

LxMLS - Lab Guide

June 18, 2026

Day 0

Transformers

0.1 Today's assignment

Today's class will demystify the Transformer architecture Vaswani et al. (2017). Transformers have become the dominant model in Natural Language Processing and Artificial Intelligence more broadly, powering models such as BERT Devlin et al. (2018), GPT Brown et al. (2020a), LLaMA Touvron et al. (2023), and the current generation of large language models (LLMs) like GPT-4 (OpenAI, 2023) and Claude. In this class, we will review the transformer architecture. The focus of the class is to understand and implement the self-attention mechanism in Pytorch. For this, we will use an annotated version of Karpathy's minGPT code (<https://github.com/karpathy/minGPT>).

0.2 Before Diving into Transformers

Before diving into transformers, we would like to introduce two important details of defining and training transformers.

0.2.1 Positional Encoding

Different from models with recurrence/convolution, positional information is not encoded in Transformers. Therefore, Vaswani et al. (2017) include the positional embedding as part of the input to Transformers. The most popular way to compute the positional embedding (PE) is to use the sinusoidal expressed as

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad (1)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad (2)$$

where pos is the position and i is the dimension. Many modern LLMs instead use learned absolute positional embeddings (as in GPT-2) or Rotary Position Embedding (RoPE) (Su et al., 2024), which encodes relative positions directly into the attention computation and is used in models such as LLaMA (Touvron et al., 2023) and Mistral (Jiang et al., 2023).

0.2.2 Tokenization

Transformers operate on sequences of input. In the case of natural language, these sequences could be whole words, single characters, or something in-between. The process of breaking a text string into these smaller pieces is called tokenization. Most transformers use subword tokenization, which operates on the idea that frequently occurring words should occur as single units, while rare words should be split into smaller pieces that occur more frequently. The most widespread subword tokenization algorithm is byte pair encoding (BPE) (Sennrich et al., 2016), which allows *any* string to be represented as a sequence of pieces from a *finite* vocabulary.

Exercise 0.1 Tokenization

Let's see how three commonly-used tokenization techniques can be applied in practice.

Authors in alphabetical order: Goncalo Melo, Grig Vardayan, Hovhannes Tamoyan, Israfil Salazar, Li Haau-Sing, Roberto Dessi, Venelin Kovachev

1. Run the word-based tokenizer and try out different text:

```
text = "I travelled to Lisbon in July to attend an NLP summer school"
text.split()
```

2. Run the character-based tokenizer and try out different different text. Can you imagine what problems character-level tokenizers pose to NLP models?

```
text = "I will travel to Lisbon in July..."
tokenized = [c for c in text if c not in [",", ";", ":", "'", "!", "?"]]
print(tokenized)
```

3. Run the BPE (byte-pair encoding) tokenizer. Can you guess which words will be split subwords and which ones won't?

```
from lxmls.transformers.bpe import BPETokenizer

tokenizer = BPETokenizer()

sentence = "Your drawing is charmingly anachronistic."
tokenizer.encoder.encode_and_show_work(sentence)
```

0.3 Transformer Architectures

The basic building block of the Transformer combines a large feed-forward network with a multi-head self-attention layer. We have seen feed-forward models in previous days and we will focus here on multi-head self-attention. Refer to previous days for details on the basics of feed-forward models. There are three main varieties of transformer model:

1. Encoders: map a sequence of T observations, e.g. some word or sub-word units $x_1 \cdots x_T$ to a hidden representation of size H , yielding a matrix of embeddings of size (H, T) . These contextual embeddings can be combined with a task-specific prediction layer, as in BERT.
2. Decoders: given a sequence of $T - 1$ observations, e.g. some word or sub-word units $x_{<T} = x_1 \cdots x_{T-1}$, predict the next word x_T . This architecture is the basis of many common large language models, such as GPT.
3. Encoder-Decoders: map a sequence x to another sequence y , which might be of a different length. For this, first x is processed with an Encoder. Then y is processed with a modified Decoder which uses a cross-attention mechanism to compute attention between x and y . Encoder-Decoder models are commonly used for machine translation, as well as in some general-purpose language models such as T5.

Although these models are used for different purposes, the architectures are very similar. In these exercises, we focus first on the Encoder and then explain the other two.

0.4 Attention

Attention mechanisms have become a crucial component in effective sequence modeling across different tasks like machine translation (Bahdanau et al., 2014), and they predate transformers by several years. The general idea of an attention mechanism is that it assigns scores to the hidden states of a model so that it knows where to focus. Figure 1 shows how an attention mechanism can decide which source words to focus on when generating each target word. In the left image, we can clearly see that the attention of the token 'cat' is specifically directed towards the token 'gato', which corresponds to its accurate translation. Similarly, in the right image,

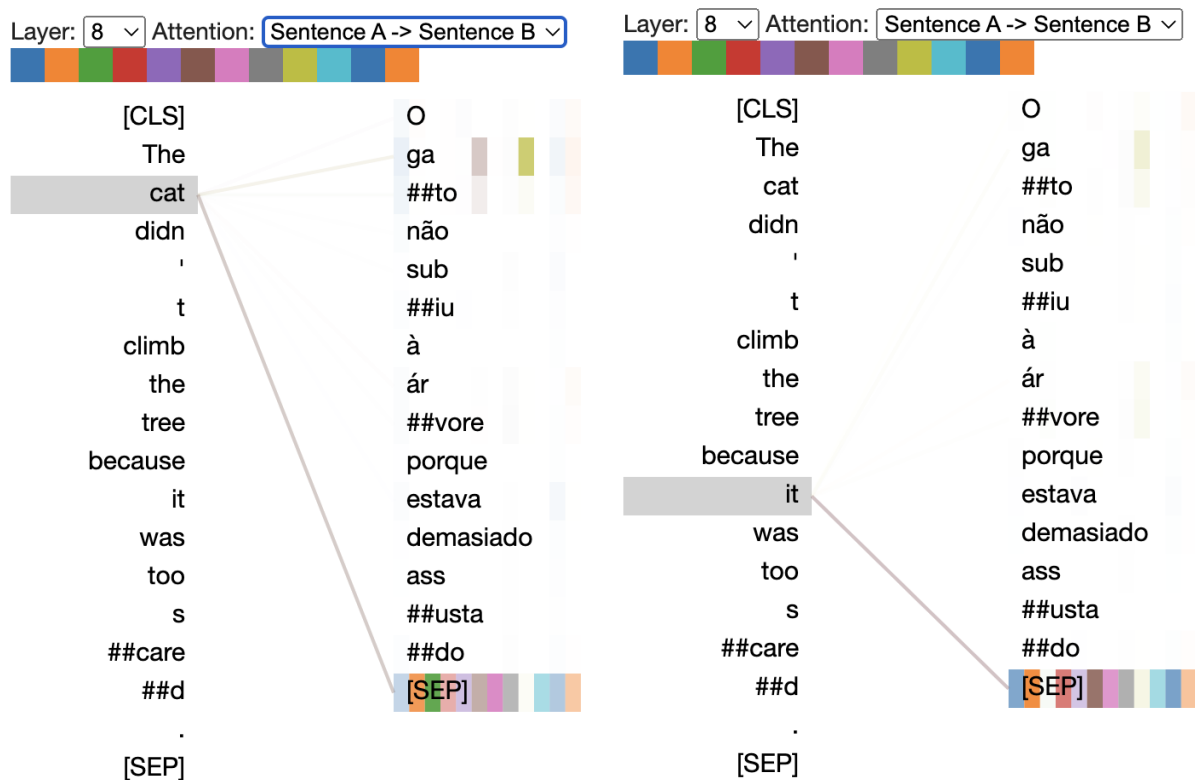


Figure 1: Encoder-Decoder Attention. Attention distributions were generated with Multilingual BERT (Devlin et al., 2018).

the word ‘it’ is attentively aligned with the token ‘gato’ once again. Transformers Vaswani et al. (2017) use several varieties of attention, most famously self-attention, in which scores are produced to model the relationship between all pairs of tokens in a single sequence (see Figure 2).

In order to understand why attention mechanisms are such powerful tools, it is helpful to understand the architectural limitations of models like RNNs. In an RNN, the hidden state h_t has a direct connection to the previous hidden state h_{t-1} , but not from all hidden states elsewhere in the sequence. This leads to information loss because all relevant information at time t needs to be put into a single fixed-dimensionality vector – information from several steps in the past may be lost, so these models tend to have a recency bias. This can be a problem in natural language applications because there are often complex relationships that depend on words far away in the sequence, as in the example in Figure 3. For a human being, the question of whether the pronoun “it” refers to the cat or the tree is relatively simple due to our prior knowledge about cats and trees. With our understanding of these concepts, we can confidently determine the intended reference of the word “it.” For a model, however, discerning the referent of the pronoun “it” – whether it is the cat or the tree – may be more of a challenge because “the tree” occurred more recently. An attention mechanism enables a model to access all hidden states $h_1 \cdots h_T$, not just the most recent.

Now let’s explore each building block of the Transformer architecture and examine how the Attention mechanism fits in this.

0.5 Encoder Architecture

In the original paper for transformers (Vaswani et al., 2017), a “sub-layer” is formed by either a Feed-Forward (FF) or Multi-Head Attention MHA (or other named sub – blocks) followed by a sequence of operations. In this section, we begin by explaining the encoder architecture. Note that the notation of “sub-layer” also applies to the following sections.

0.5.1 Simplified Encoder Architecture

The encoder architecture can be succinctly described as stacking a number of N blocks on top of each other combining a FF and MHA sub-blocks. A single block is defined as

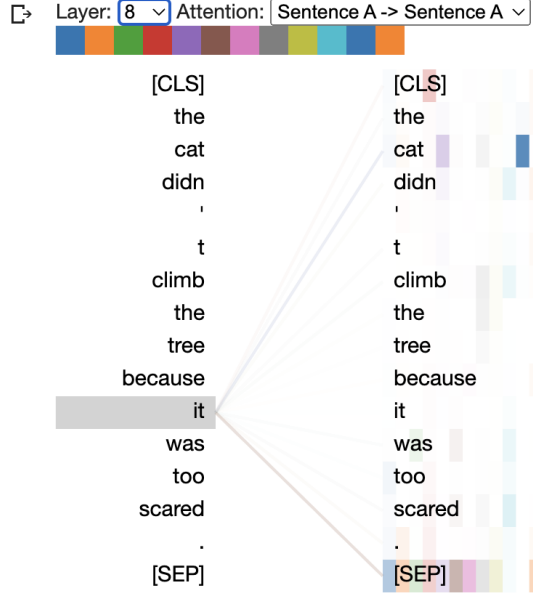


Figure 2: Self-Attention

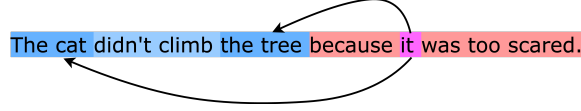


Figure 3: An example illustration

$$e^{n+1} = \text{FF}((\text{MHA}(e^n))). \quad (3)$$

The input to the first block e^0 is the sum of position P and non-contextual embeddings E of the input. Assuming $x_1 \cdots x_T$ is a sequence of integers (indices to a vocabulary of V symbols) we have that

$$e_t^0 = E \cdot 1_{x_t} + P \cdot 1_t \quad \text{for } t = 1 \cdots T \quad (4)$$

where 1_{x_t} and 1_t are indicators, i.e. one-hot, vectors for the token content (vocabulary symbol) and the token position. $E \in \mathbb{R}^{H \times V}$ is the non-contextual embedding matrix for each symbol in the vocabulary and $P^{H \times \tau}$ is the position embedding matrix, where $\tau - 1$ is the furthest position supported. See Figure 4 for the construction of the input of the first block.

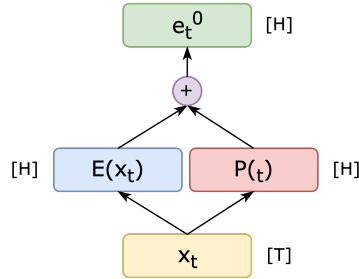


Figure 4: Construction of the input of the t -th token in layer e^0

The feed-forward (FF) is given by:

$$\text{FF}(z) = W^2 \cdot \text{gelu}(W^1 \cdot z) \quad (5)$$

with weight matrices $W^2 \in \mathbb{R}^{H \times 4H}$ and $W^1 \in \mathbb{R}^{4H \times H}$, that expand and contract the hidden dimension H .

Attention consists of three matrices: Query (Q), Key (K), and Value (V). To obtain Q , K , and V , we perform a linear projection of the input using corresponding matrices that are learned during the training process.

The concept of Query, Key, and Value is similar to retrieval systems. When searching for an article on the web using a specific Query, the search engine maps the Query against a set of Keys or titles associated with candidate results in their database. It then presents the best-matched articles or Values to the user. In concrete terms, we can think of it as a weighted modification of a query Q , given some context K and V . There are two cases to consider:

1. Cross-Attention: The query $Q = W^Q \cdot z_q$ is a query from one sequence, and $K = W^K \cdot z_c$ and $V = W^V \cdot z_c$ are the context from another sequence.
2. Self-Attention: The query $Q = W^Q \cdot z$ is a query from one sequence, and $K = W^K \cdot z$ and $V = W^V \cdot z$ are the context from the same sequence.

In Figure 5, we present the computation for K , Q , and V in a cross-attention scenario with a given input z_q and a context z_c . Notice that this case is more general, for self-attention, we have $z_q = z_c = z$.

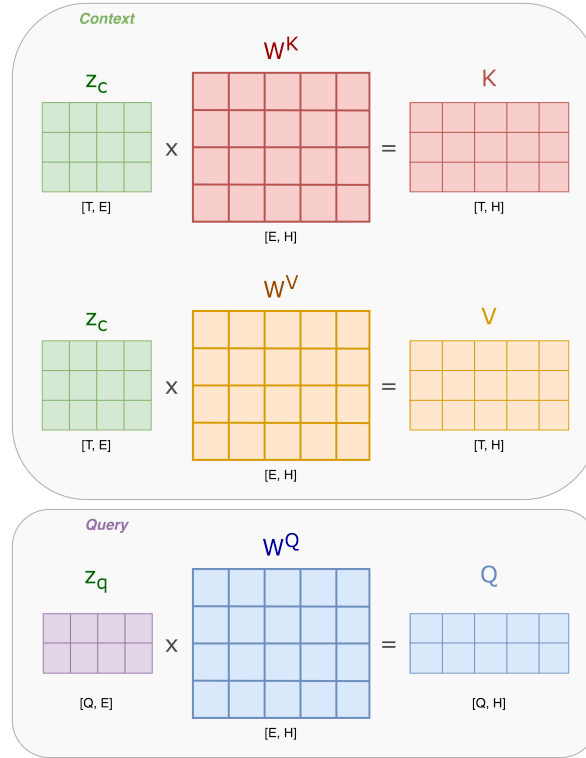


Figure 5: Query, Key, and Value projections from a given query z_q , and context z_c .

Then, to perform attention, we follow these steps:

1. Measure Similarity: We calculate the similarity between, Query and Key. This is often done by taking the dot product of the two matrices.
2. Find Maximum Match: We extract the key with the maximum match. This means identifying the Key that has the highest similarity with the Query.
3. Retrieve Value: Once we have the Key with the maximum match, we can retrieve the corresponding Value associated with that Key.

This simple 3-step process is in fact the Attention mechanism. Where we are looking for the best matching tokens for the source sequence in the target sequence. Let's compute the cross-attention score step-by-step. The Key and Query matrices are multiplied by each other and normalized by a constant value $\frac{1}{A} W^Q z_q (W^K z_c)^T$ or $\frac{1}{A} Q K^T$. The A value is usually chosen to be the square root of the dimension of the key matrices. This leads to having more stable gradients.

Then pass the result through a *softmax* operation. The Softmax normalizes the scores so they're all positive and add up to 1. In other words, extracts a distribution of relative scores from a given token for each token in the input sequence.

After this multiply the output by the Value matrix: $\text{softmax}_{i \rightarrow} \left(\frac{1}{A} W^Q z (W^K z)^T \right) \cdot W^V \cdot z$ or simply $V \cdot \text{softmax}_{i \rightarrow} \left(\frac{1}{A} Q K^T \right)$.
See Figure 6

$$Attention(Q, K, V) = \text{softmax} \left(\frac{Q \times K^T}{A} \right) \times V$$

Figure 6: The attention mechanism

The Multi-Head Attention (MHA) enhances the Attention layer in two ways: it allows the model to focus on different positions within the input, enabling a better understanding of pronoun references; and allowing projection of input embeddings into distinct representation subspaces, thereby improving overall performance. Finally getting the following mathematical form for the MHA:

$$MHA(z) = W^O \cdot \begin{bmatrix} \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_1^Q z (W_1^K z)^T \right) \cdot W_1^V \cdot z \\ \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_2^Q z (W_2^K z)^T \right) \cdot W_2^V \cdot z \\ \dots \\ \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_D^Q z (W_D^K z)^T \right) \cdot W_D^V \cdot z \end{bmatrix}$$

where we have D attention heads. Each head contracts the hidden dimension $W^K, W^Q, W^V \in \mathbb{R}^{E \times H}$ into a space of size E which is equal to H/D (this has practical implementation consequences, H and D should be set properly). Outputs of all heads are concatenated and projected again with $W^O \in \mathbb{R}^{H \times H}$. Please note that the dimension of W^V can be different in the general case. The above-mentioned dimensions are correct for the cases when the source and target sequences have equal lengths, which is mostly not the case.

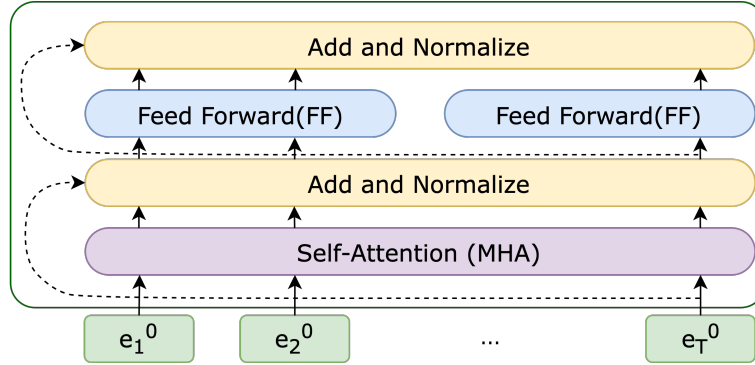


Figure 7: Encoder block

0.5.2 Adding Dropout, Residuals, and Layer Normalization

To complete the “sub-layer”, a dropout followed by a residual connection and a layer normalization is applied to input z that has been passed through a sub – block, where a sub – block $\in \{FF, MHA\}$. Together, we can express the function as

$$C(x) = \text{layernorm}(\text{residual}(\text{dropout}(\text{sublayer}(z))), z) \quad (6)$$

where $\text{sublayer} \in \{FF, MHA\}$.

Specifically, we have

- $\text{dropout}(x) = r * x$ where $r_j \sim \text{Bernoulli}(p)$ and $*$ refers to element-wise multiplication,
- $\text{residual}(x, z) = x + z$, and

Note that the default p is 0.1.

- $\text{layernorm}(x) = \frac{x - E(x)}{\sqrt{\text{Var}(x) + \epsilon}} * \gamma + \beta$ where $E(x)$ and $\text{Var}(x)$ are computed among dimensions other than the dimension for the batch.

After incorporating all of these subcomponents, we obtain the final Encoder block, as illustrated in Figure 7

0.6 Decoder Architecture

This is identical to the Encoder architecture with two differences

1. We feed if a partial sequence $x_{<t}$ and take the last output h_{t-1} as the hidden vector for x_t
2. We mask every head of MHA to prevent any value of time p to depend on values of $> p$

The implementation of training realizes 1) by masking input partial sequence $x_{<t}$ and hidden units from the corresponding positions with an attention mask. This attention mask is also applied during inference time.

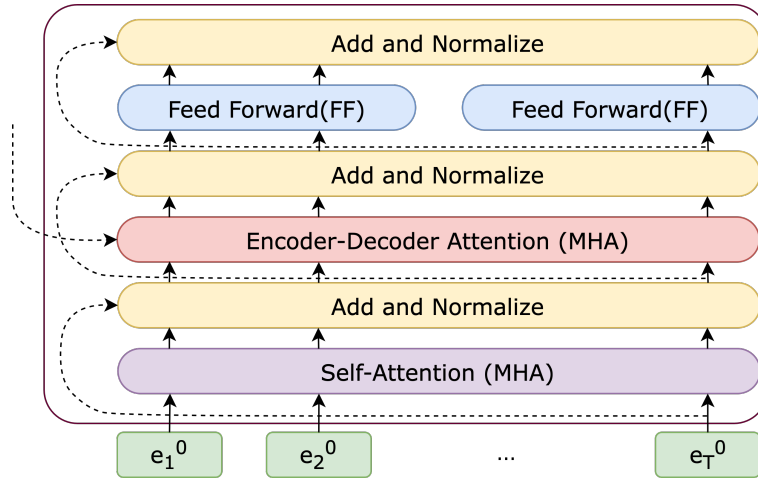


Figure 8: Decoder block

Once all these subcomponents are integrated, we achieve the final Decoder block, which is depicted in Figure 8.

0.7 Encoder-Decoder Architecture

In the Encoder-Decoder Architecture, the encoder is the same as section 0.5. Therefore, we have the encoded embeddings as $e^N = \text{Encoder}(x)$.

The decoder in the Encoder-Decoder Architecture is a little bit different than section 0.6. Each layer of the decoder will include one more Cross-Multi-Head Attention (CMHA) sub-layer, and the order of sub-layers in a decoder layer will be MHA-CMHA-FF, i.e.

$$d^{m+1} = \text{FF}((\text{CMHA}(e^N, \text{MHA}(d^m))). \quad (7)$$

Specifically, the query matrix is computed from the layer below it, while the key and value matrix are computed from e^N , which can be expressed as

$$\text{CMHA}(e^N, \text{MHA}(d^m)) = W^o \cdot \begin{bmatrix} \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_1^Q \text{MHA}(d^m) (W_1^K e^N)^T \right) \cdot W_1^V \cdot e^N \\ \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_2^Q \text{MHA}(d^m) (W_2^K e^N)^T \right) \cdot W_2^V \cdot e^N \\ \vdots \\ \text{softmax}_{i \rightarrow} \left(\frac{1}{A} W_D^Q \text{MHA}(d^m) (W_D^K e^N)^T \right) \cdot W_D^V \cdot e^N \end{bmatrix}.$$

By combining N blocks of Encoder and Decoder, we obtain the complete view of the Transformer architecture, as illustrated in Figure 9.

Note that the default ϵ is $1e - 5$. γ and β are learnable affine parameters if we want to learn, otherwise set to 1 and 0

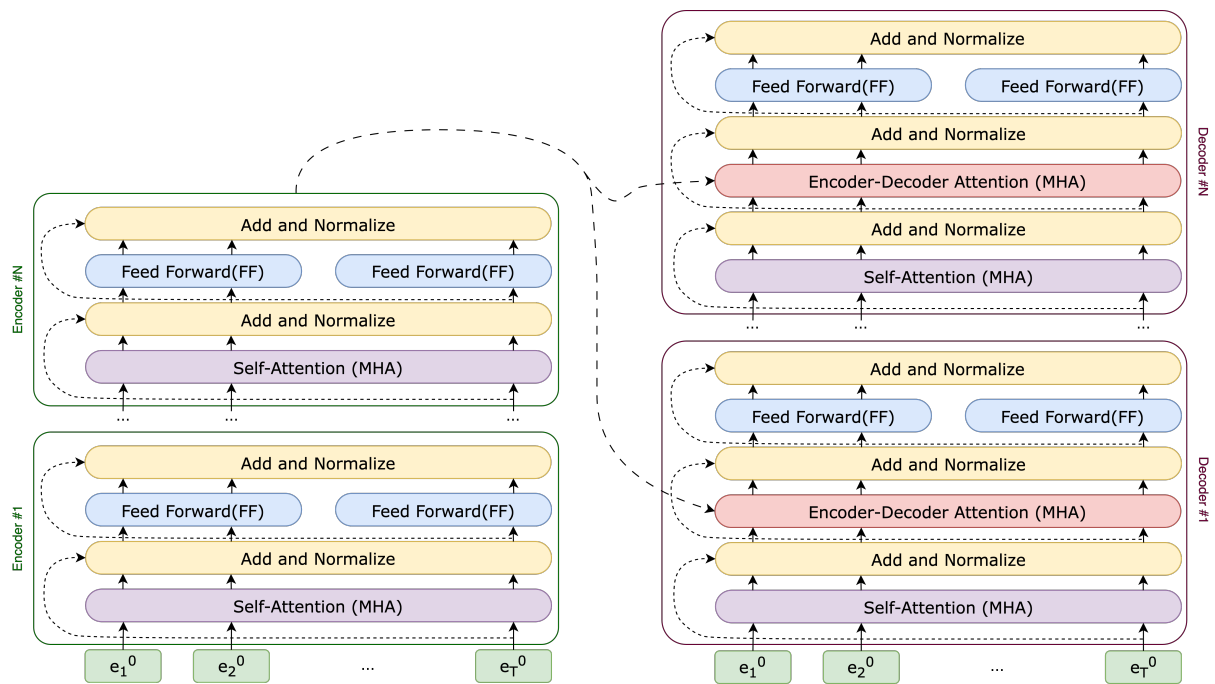


Figure 9: Encoder-Decoder Architecture

Exercise 0.2 *Cross Multi-Head Attention & Multi-Head Attention* Now let's implement our own Attention mechanism, to that end let's:

- Complete the `cross_attention` function. Given two input sequences S_1 and S_2 and the transformation weights W_Q , W_K , and W_V . You need to implement the following:
 - Calculate the query, key, and value projections using linear transformations.
 - Compute the attention scores by performing the dot product between the query and key tensors.
 - Apply softmax activation to the attention scores to obtain the attention weights.
 - Multiply the attention weights with the value tensor to get the attended values.
 - Return the attended values.

```
import torch
import torch.nn.functional as F

def cross_attention(S1, S2, W_Q, W_K, W_V):
    ## Your code here
    return attended_values
```

- Complete the `CausalSelfAttention` class. First, you should create linear projections `query_proj`, `key_proj` and `value_proj`.

```
import math
import torch.nn as nn

class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()

        # Initialize layers and parameters
        self.hidden_size = config.n_embd
        self.num_heads = config.n_head

        # TODO: Create the linear projections
```

```

self.output_proj = nn.Linear(config.n_embd, config.n_embd)

self.attn_dropout = nn.Dropout(config.attn_pdrop)
self.resid_dropout = nn.Dropout(config.resid_pdrop)

self.register_buffer(
    "bias",
    torch.tril(torch.ones(config.block_size, config.block_size)).view(
        1, 1, config.block_size, config.block_size))

```

Then apply query_proj, key_proj and value_proj to split input into query, key, and value tensors:

```

def forward(self, x: torch.Tensor) -> torch.Tensor:
    B, T, C = x.size()

    # TODO: Split input into query, key, and value tensors

    # Reshape and transpose tensors for multi-head computation
    query = query.view(B, T, self.num_heads,
                       self.hidden_size // self.num_heads).transpose(1, 2)
    key = key.view(B, T, self.num_heads,
                  self.hidden_size // self.num_heads).transpose(1, 2)
    value = value.view(B, T, self.num_heads,
                      self.hidden_size // self.num_heads).transpose(1, 2)

```

Then compute the attention scores. Note that the shape of scores should be (B, num_heads, T, T)

```

# TODO: Compute attention scores

# Apply a causal mask to restrict attention to the left in the input
sequence
mask = self.bias[:, :, :T, :T]
scores = scores.masked_fill(mask == 0, float('-inf'))

```

Finally, apply soft-max activation and multiply attention weights with values to get attended values:

```

# TODO: Apply softmax activation to get attention weights

# TODO: Multiply attention weights with values to get attended values

# Transpose and reshape attended values to restore original shape
attended_values = attended_values.transpose(1, 2).contiguous().view(
    B, T, C)

# Apply output projection and dropout
output = self.resid_dropout(self.output_proj(attended_values))

return output

```

0.8 Training different Transformers

We have introduced different architectures of transformers and will present different variants of transformers that are trained with different objectives, and thus applied to different downstream tasks. We briefly introduce all these transformers and provide references so you can read these papers if you are interested.

0.8.1 Training with Encoders

Encoder models solely utilize the encoder component of a Transformer model. In each step, the attention layers have the ability to consider all the words present in the original sentence. These models are known as 'auto-encoding models' and are recognized for their 'bi-directional' attention mechanism.

The pretraining of these models typically involves manipulating a given sentence, often by obscuring random words and assigning the model the task of identifying or reconstructing the original sentence. This objective is commonly referred to as Masked Language Modeling (MLM), and models pre-trained with this objective are known as Masked Language Models.

For a text sequence $\mathbf{x}$, the BERT model first constructs a corrupted version $\hat{\mathbf{x}}$. Let the masked tokens be $\bar{\mathbf{x}}$. The training objective is to reconstruct $\bar{\mathbf{x}}$ from $\hat{\mathbf{x}}$:

$$\max_{\theta} \log P_{\theta}(\bar{\mathbf{x}}|\hat{\mathbf{x}}) \approx \sum_{t=1}^T m_t \log P_{\theta}(x_t|\hat{\mathbf{x}}) = \sum_{t=1}^T m_t \log \frac{\exp(H_{\theta}(\hat{\mathbf{x}})_t^T e(x_t))}{\sum_{x'} \exp(H_{\theta}(\hat{\mathbf{x}})_t^T e(x'))} \quad (8)$$

Where the $\approx$ indicates that all $\bar{\mathbf{x}}$ elements are independent (seperately reconstructed), m_t indicates whether or not x_t is masked (1-masked, 0-not), H_{θ} is a Transformer that maps a sequence x of length T and contains information about the context on both sides $H_{\theta}(X) = [H_{\theta}(x)_1, H_{\theta}(x)_2, \dots, H_{\theta}(x)_T]$, and $e(x)$ denotes the embedding of x .

Encoder models excel in tasks that demand a comprehensive understanding of the entire sentence. These tasks include sentence classification, word classification tasks e.g. named entity recognition, and extractive question answering. Widely used representatives of this model family include BERT Devlin et al. (2018) and RoBERTa Liu et al. (2019). Note that while encoder-only models remain useful for classification and retrieval tasks, the field has largely shifted toward decoder-only LLMs for general-purpose language understanding.

BERT Devlin et al. (2018) is trained on MLM(Masked Language Modeling) and NSP(Next Sentence Prediction) objectives. In the **MLM** objective, 15% of the input tokens are selected. Among these selected tokens:

- 80% of the time, the mask token is inserted in place of the original token.
- 10% of the time, a random token is inserted in place of the original token.
- 10% of the time, the original token remains unchanged.

The **NSP** objective is applied to pairs of sentences, A and B, taken from the training set. In 50% of the cases, sentence B directly follows sentence A in the input, while in other cases, the pairs are randomly selected. The objective is to perform binary classification and predict whether sentence B follows sentence A or not.

RoBERTa Liu et al. (2019) builds upon BERT's pre-training by addressing its perceived undertraining. It introduces the following modifications:

- Longer and larger-scale training: RoBERTa trains the model for an extended period using larger batches, more data, and longer sequences.
- Removal of NSP objective: The next sentence prediction (NSP) objective, present in BERT, is eliminated in RoBERTa.
- Dynamic masking: RoBERTa employs dynamic masking by duplicating the training data ten times and applying different mask patterns to each sample. This contrasts with BERT's static masking, where a fixed mask is used for each sample.

These modifications aim to enhance RoBERTa's pre-training performance and overall language understanding capabilities.

0.8.2 Training with Decoders

Decoder-only or Autoregressive modeling performs pretraining by maximizing the likelihood under the forward autoregressive factorization:

$$\max_{\theta} \log P_{\theta}(\mathbf{x}) = \sum_{t=1}^T \log P_{\theta}(x_t|\mathbf{x}_{<t}) = \sum_{t=1}^T \log \frac{\exp(h_{\theta}(\mathbf{x}_{1:t-1})^T e(x_t))}{\sum_{x'} \exp(h_{\theta}(\mathbf{x}_{1:t-1})^T e(x'))}$$

Where the $h_{\theta}(\mathbf{x}_{1:t-1})$ is the context representation produced by the neural model, and it contains information (conditioned) about the tokens up to position t , and $e(x_t)$ is the embedding of a token x_t .

A Decoder-only or Autoregressive model operates differently compared to Encoder-only or Autoencoder model by focusing on density estimation rather than reconstructing corrupted input. These models aim to estimate probability distributions, which limits their ability to capture bidirectional context. As a result, they are restricted to uni-directional processing.

GPTs. The class of GPTs is a series of pre-trained decoder-only transformers. Models are pre-trained to perform the next token prediction with the Cross-Entropy criterion. Since the release of GPT-1 (Radford et al., 2018), GPTs are being trained with more parameters and training data, with GPT-2 (Radford et al., 2019), GPT-3 (Brown et al., 2020b), InstructGPT (Ouyang et al., 2022) and GPT-4 (OpenAI, 2023) being subsequently released. Note that since the release of InstructGPT (Ouyang et al., 2022), this class of language models has been trained with reinforcement learning from human feedback (RLHF) (Bai et al., 2022), which enables the model to follow instructions and exhibit the behaviors you see when using ChatGPT.

Beyond OpenAI's models, the release of LLaMA (Touvron et al., 2023) and its successors (LLaMA 2, LLaMA 3) made high-quality decoder-only transformers available to the research community. Models such as Mistral (Jiang et al., 2023) further demonstrated that smaller, carefully trained models can match much larger ones. These open-weight models have become the dominant platform for NLP research.

0.8.3 Training with Encoder-Decoder

To leverage the strengths of both Encoder-only and Decoder-only models, the concept of Encoder-Decoder models was introduced. While previous methods effectively captured bidirectional information for text generation, they had certain limitations in terms of contextual token representations. One common approach involved concatenating the left-to-right and right-to-left representations Peters et al. (2018).

Encoder-Decoder models aim to overcome these limitations by combining the advantages of both approaches. These models can effectively capture bidirectional information while maintaining robust contextual representations for each token. By leveraging the strengths of both encoders and decoders, Encoder-Decoder models offer enhanced capabilities for various natural language processing tasks, including text generation.

Commonly employed members of this model family include BART Lewis et al. (2020) and T5 Raffel et al. (2020), which have gained significant popularity in the field.

BART. BART Lewis et al. (2020) an encoder-decoder model pre-trained on five tasks injecting noises into the input text: i) token masking (same as BERT Devlin et al. (2018)), ii) token deletion, iii) text infilling by replacing sampled input spans with single masks, iv) sentence permutation, and v) document rotation. The powerful pre-trained denoising autoencoder is commonly used in generation tasks.

T5. T5 (Raffel et al., 2020) utilizes a text-to-text methodology. In this approach, various tasks like translation, question answering, and classification are transformed into a unified format. The model is provided with input text and trained to generate the corresponding output text. To achieve this, a task-specific prefix(instruction) is added to the input sequence, and the model is pre-trained to produce outputs specific to each task.

Exercise 0.3 Training Transformers

Let's delve into practical training of a Transformer model to gain hands-on experience. Our initial step will involve creating a Decoder-based model (GPT-2 Radford et al. (2019)) and training it on a modest dataset. Once this is completed, we can visualize the attention mechanism before and after the training process. Finally, we can compare the performance of the trained model during inference with that of the smaller trained model.

Exercise 0.4 Training a Weather Prediction Model Using Autoregressive Transformer.

In this exercise, we will be working with a dummy weather dataset comprising sequences of weather observations and their corresponding states. The objective is to train a compact autoregressive transformer model that can predict the weather state based on previous observations. Let's begin by importing the required modules and classes, and setting a seed value to ensure consistent output every time we run the file.

```
import random
import time

import numpy as np
import torch
from torch.utils.data.dataloader import DataLoader
```

```

random.seed(42)

from lxmls.transformers.bpe import BPETokenizer
from lxmls.transformers.dataset import WeatherDataset
from lxmls.transformers.model import GPT
from lxmls.transformers.trainer import Trainer
from lxmls.transformers.utils import set_seed

```

To begin, we initialize the training dataset, which comprises sequences of weather observations along with their corresponding states. These sequences are transformed into indices and then concatenated to create the input and output sequences for the transformer model. For more information on this, refer to: "[lxmls/transformers/dataset.py](#)".

```

fixed_proba = {}
fixed_proba["initial"] = [0.5, 0.3, 0.2]
fixed_proba["transition"] = [[0.5, 0.5, 0], [0, 0.5, 0.5], [0.5, 0, 0.5]]
fixed_proba["emission"] = [
    [0.5, 0, 0.2, 0, 0.3],
    [0, 0.5, 0.4, 0, 0.1],
    [0, 0, 0.1, 0.5, 0.4],
]

```

Let's print an example instance of the dataset.

```

train_dataset = WeatherDataset("train", proba=fixed_proba)
test_dataset = WeatherDataset("test", proba=train_dataset.proba)
x, y = train_dataset[0]

print("Sampling from the dataset:")
print(f"Input: {train_dataset.decode_obs(x.tolist()[:6])}")
print(f"Labels: {train_dataset.decode_st(y.tolist()[5:])}")
print("-" * 50)
print("Tokenized sequences:")
print(f"Input: {x.tolist()}")
print(f"Labels: {y.tolist()}")

```

Moving forward, we construct a model using the default configuration for the GPT model, which encompasses parameters defining the model's size and structure. In this case, we employ a compact variant known as GPT Nano.

```

model_config = GPT.get_default_config()
model_config.model_type = "gpt-nano"
model_config.vocab_size = train_dataset.get_vocab_size()
model_config.block_size = train_dataset.get_block_size()
model = GPT(model_config)

print(model_config)

```

To facilitate the training of our model, we instantiate a Trainer object. The Trainer manages various aspects of the training process, such as configuring the learning rate, specifying the maximum number of iterations, and determining the number of workers for data loading. We initialize the Trainer with the model, training dataset, and validation dataset.

```

train_config = Trainer.get_default_config()
train_config.learning_rate = (
    5e-4 # The model we're using is so small that we can go a bit faster
)
train_config.max_iters = 2000
train_config.num_workers = 0
train_config.device = "cuda" if torch.cuda.is_available() else "cpu"
trainer = Trainer(train_config, model, train_dataset)

```

```
print(train_config)
```

With all these components in position, we are fully prepared to train our model using the weather dataset and leverage the acquired patterns to generate predictions.

```
def batch_end_callback(trainer):
    if trainer.iter_num % 100 == 0:
        print(
            f"iter_dt {trainer.iter_dt * 1000:.2f}ms; iter {trainer.iter_num}: train loss "
            f"{trainer.loss.item():.5f}"
        )

trainer.set_callback("on_batch_end", batch_end_callback)

start_time = time.time()
trainer.run()
end_time = time.time()
elapsed_time = end_time - start_time

# Print the training time
print("Training time: {:.2f} seconds".format(elapsed_time))
```

Now, let's visualize the attention heads of our trained model. To do this, we need to install the HF transformers and bertviz packages. You can install them by running the following command: "pip install transformers bertviz".

```
from transformers import BertTokenizer, BertModel
from bertviz import head_view

# Define a sample input text
text = "I will go for a run and will jump into a lake."

# Instantiate the BERT tokenizer and model
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

# Tokenize the input text
tokens = tokenizer.tokenize(text)

# Convert tokens to token IDs
token_ids = tokenizer.convert_tokens_to_ids(tokens)

# Create attention mask
attention_mask = [1] * len(token_ids)

# Convert token IDs and attention mask to tensors
input_ids = torch.tensor([token_ids])
attention_mask = torch.tensor([attention_mask])

# Generate the transformer output
outputs = model(input_ids, attention_mask=attention_mask, output_attentions=True)

head_view(outputs.attentions, tokens=tokens)
```

Exercise 0.5 Prompting a GPT-2 model Now that we have trained our own Transformer model and gained some understanding of its functioning, let's explore a larger pre-trained model that has been trained on a vast amount of natural data. By doing so, we can experience generating output that closely resembles human-like responses.

```
model_type = "gpt2"
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
model = GPT.from_pretrained(model_type)

# move model to the device (GPU if available)
# set to eval mode to avoid gradient accumulation
model.to(device)
model.eval()

# Random prompt, uses pooling
for i in range(5):
    set_seed(42)
    model.prompt("Alan Turing, the", 50, 3)

# Deterministic prompt, does NOT use pooling
for i in range(5):
    model.prompt_topK("Alan Turing, the", 50, 3)
```

Get ready to embark on an intriguing journey by playing with the input prompt and witnessing the fascinating and captivating outputs that await you!

Bibliography

- Bahdanau, D., Cho, K., and Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., Johnston, S., Kravec, S., Lovitt, L., Nanda, N., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., Mann, B., and Kaplan, J. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020a). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020b). Language models are few-shot learners. *CoRR*, abs/2005.14165.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. (2023). Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., and Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach.
- OpenAI (2023). Gpt-4 technical report.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. (2018). Deep contextualized word representations.
- Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. (2018). Improving language understanding by generative pre-training.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer.

- Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.